Certificación de Entrega — Entrega 5: Seguridad y confiabilidad

Módulo 5 — Seguridad y confiabilidad en entornos distribuidos

> Qué evalúa esta entrega: que el grupo haya pensado qué pasa cuando algo falla o es atacado, no solo el camino feliz.
>
> **Estado al momento:** el checklist refleja lo que el repositorio hace hoy (rama `pdc/santiago-godoy`). Solo la autenticación está implementada. Para el resto se deja una **propuesta** de cómo quedaría en la próxima entrega, sin marcarlo como cumplido.

Datos generales

Campo	Valor
Grupo N°	*Completar* (Godoy – Iriarte – Lagoria)
Tag / Release de esta entrega	*Completar* (propuesto: `v5-seguridad`)
Fecha de entrega	*dd/mm/aaaa*

Checklist de la entrega

Requisito	Cumple	Justificación breve / evidencia
Hay autenticación de usuarios implementada.	**Sí**	`cliente-api/app/auth.py`: login contra Postgres (`POST /auth/login`), contraseñas con bcrypt, JWT HS256 con vencimiento de 12 h. Todos los endpoints de `/imagenes` exigen token válido (`Depends(auth.usuario_actual)`). Usuarios cargados con `crear_usuarios.py`.
Hay autorización / control de acceso implementado (no todos los usuarios pueden todo).	**No**	La tabla `usuarios` ya guarda `rol` (`admin` / `colaborador`) y viaja dentro del JWT, pero ningún endpoint lo consulta: todo usuario autenticado puede subir, ver y analizar todas las imágenes. **Propuesta:** ver §A.
La comunicación entre componentes está cifrada en tránsito (HTTPS/TLS).	**No**	El nginx de `cliente-web` escucha en el puerto 80 (HTTP) y no hay certificados. El tráfico navegador → gateway → API → Postgres/IA viaja sin cifrar. **Propuesta:** ver §B.
Hay un mecanismo de replicación o recuperación ante fallos implementado y probado.	**No**	Hoy solo hay `restart: unless-stopped` en los servicios y un `healthcheck` de Postgres, que reinician procesos caídos pero no recuperan datos perdidos. No hay backup, réplica ni prueba de recuperación. **Propuesta:** ver §C.

Requisito	Cumple	Justificación breve / evidencia
Se incluyó un informe corto de amenazas consideradas (qué se protegió y de qué).	**No**	Todavía no existe el informe. Sí se relevaron las debilidades actuales (§D) como insumo. **Propuesta:** ver §D.

Propuestas para la próxima entrega

A. Autorización por rol

- Dependency `require_rol("admin")` en `auth.py`, apoyada en el `rol` que ya viaja en el JWT.
- Permisos propuestos:

Acción	`admin`	`colaborador`
Subir ortomosaico	Sí	Sí
Ver / analizar imágenes propias	Sí	Sí
Ver imágenes de otros usuarios	Sí	No
Borrar imágenes	Sí	Solo las propias
Administrar usuarios	Sí	No

- Guardar el `propietario` en `meta.json` de cada imagen y filtrar `GET /imagenes` por él.
- Prueba: script con dos tokens que verifica `403` al acceder a una imagen ajena.

B. Cifrado en tránsito (TLS)

- nginx de `cliente-web` escucha en `443` con certificado y redirige `80 → 443`. Desarrollo: certificado autofirmado generado con `openssl`; despliegue real: Let's Encrypt si hay dominio.
- `cliente-api` e `ia-inferencia` dejan de publicar puertos al host (`8000`, `8001`, `5432`): el único punto de entrada es el gateway por HTTPS.
- Postgres con `sslmode=require` en `DATABASE_URL`.
- Prueba: `curl -I http://...` devuelve `301`; `curl -k https://...` devuelve `200`.

C. Recuperación ante fallos

- Script `backup.sh` con `pg_dump` de Postgres y copia de `datos/ortomosaicos`, con retención (por ejemplo, los últimos 7) y ejecución diaria con cron.
- Script `restaurar.sh` que recrea la base desde el último backup.
- Prueba reproducible, a la manera de `probar-nodos.sh`: crear usuario e imagen → backup → destruir el volumen de la base → restaurar → verificar que el login y el listado siguen funcionando. Se guarda la salida en `salida-prueba.txt`.
- Opcional si sobra tiempo: réplica de streaming de Postgres en un segundo contenedor.
- Sumar `healthcheck` a `cliente-api` e `ia-inferencia` (hoy solo lo tiene `db`) y `depends_on: condition: service_healthy`.

D. Informe de amenazas

Tabla STRIDE sobre lo que existe, con **qué se protege**, **de qué** y **cómo**. Debilidades relevadas hoy que el informe debe cubrir:

Debilidad actual	Amenaza	Mitigación propuesta
`JWT_SECRET` con valor por defecto en el código y en los compose	Falsificación de tokens	Exigir la variable en producción; fallar al arrancar si falta
Token aceptado también como `?token=` (necesario para teselas y SSE)	Fuga del token en logs y en el historial	Cookie `HttpOnly` o tokens de corta vida solo para teselas
Contraseñas iniciales escritas en `crear_usuarios.py`	Credenciales en el historial de git	Leerlas de variables de entorno; forzar cambio en el primer ingreso
Contraseña de Postgres `agrovision/agrovision` por defecto y puerto 5432 expuesto en `compose.dev.yml`	Acceso directo a la base	Secreto por `.env`; no publicar el puerto
Sin límite de intentos de login	Fuerza bruta	Rate limiting en nginx o en la API
Sin autorización por rol ni por propietario	Acceso a datos ajenos	§A
HTTP plano	Intercepción y manipulación	§B
Sin backups	Pérdida de datos por falla o borrado	§C
Subida de archivos de hasta 8 GB sin cuota	Denegación de servicio por disco	Cuota por usuario

Evidencia adicional

- Código de la autenticación: `project/servicios/cliente-api/app/auth.py`
- Informe de amenazas y prueba de recuperación ante fallos: *pendientes para la próxima entrega* (§C y §D).

Espacio para el docente

Nota	Corregido por	Fecha de corrección
___ / 10		__ /__ / ____

Observaciones: